
SWS3009B – Tele-Robotic

Arduino meets Pi

Outline

■ **Serial communications**

- ❑ Raspberry Pi Configuration
- ❑ Level Shifter
- ❑ Connection

■ **Communication Protocol**

- ❑ Handshake
- ❑ Push / Pull
- ❑ Data Packet Format

Pi:"Hello Arduino!" **Arduino:**"Hey there, Pi!"

PI – ARDUINO COMMUNICATION

Relook at Serial Communication

- We have been using serial communication in Arduino Mega:
 - ❑ Through serial port 0 for input / output / debugging purpose
- On both Arduino Mega and Raspberry Pi
 - ❑ Serial communication is enabled by UART
 - ❑ UART = Universal Asynchronous Receiver/Transmitter
 - ❑ So, when we say UART ports, we are referring to the serial ports on these platforms

UART ports on **Arduino Mega**

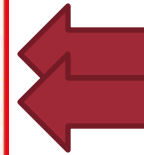
- Arduino Mega support **4 sets of serial communication** (two ways each):
 - ❑ **Serial 0**: Through the USB connection or Pin 0 (Receive RX) + Pin 1 (Transmit TX)
 - ❑ **Serial 1**: Pin 19 (RX) + Pin 18 (TX),
 - ❑ **Serial 2**: Pin 17 (RX) + Pin 16 (TX)
 - ❑ **Serial 3**: Pin 15 (RX) + Pin 14 (TX)

Arduino Mega operates at 5volt

UART on the Raspberry Pi



3.3V PWR	1		2	5V PWR
I2C1 SDA	3		4	5V PWR
I2C1 SCL	5		6	GND
GPIO 4	7		8	UART0 TX
GND	9		10	UART0 RX
GPIO 17	11		12	GPIO 18
GPIO 27	13		14	GND
GPIO 22	15		16	GPIO 23
3.3V PWR	17		18	GPIO 24
SPI0 MOSI	19		20	GND
SPI0 MISO	21		22	GPIO 25
SPI0 SCLK	23		24	SPI0 CS0
GND	25		26	SPI0 CS1
Reserved	27		28	Reserved
GPIO 5	29		30	GND
GPIO 6	31		32	GPIO 12
GPIO 13	33		34	GND
GPIO 19	35		36	GPIO 16
GPIO 26	37		38	GPIO 20
GND	39		40	GPIO 21



Raspberry Pi operates at 3.3volt

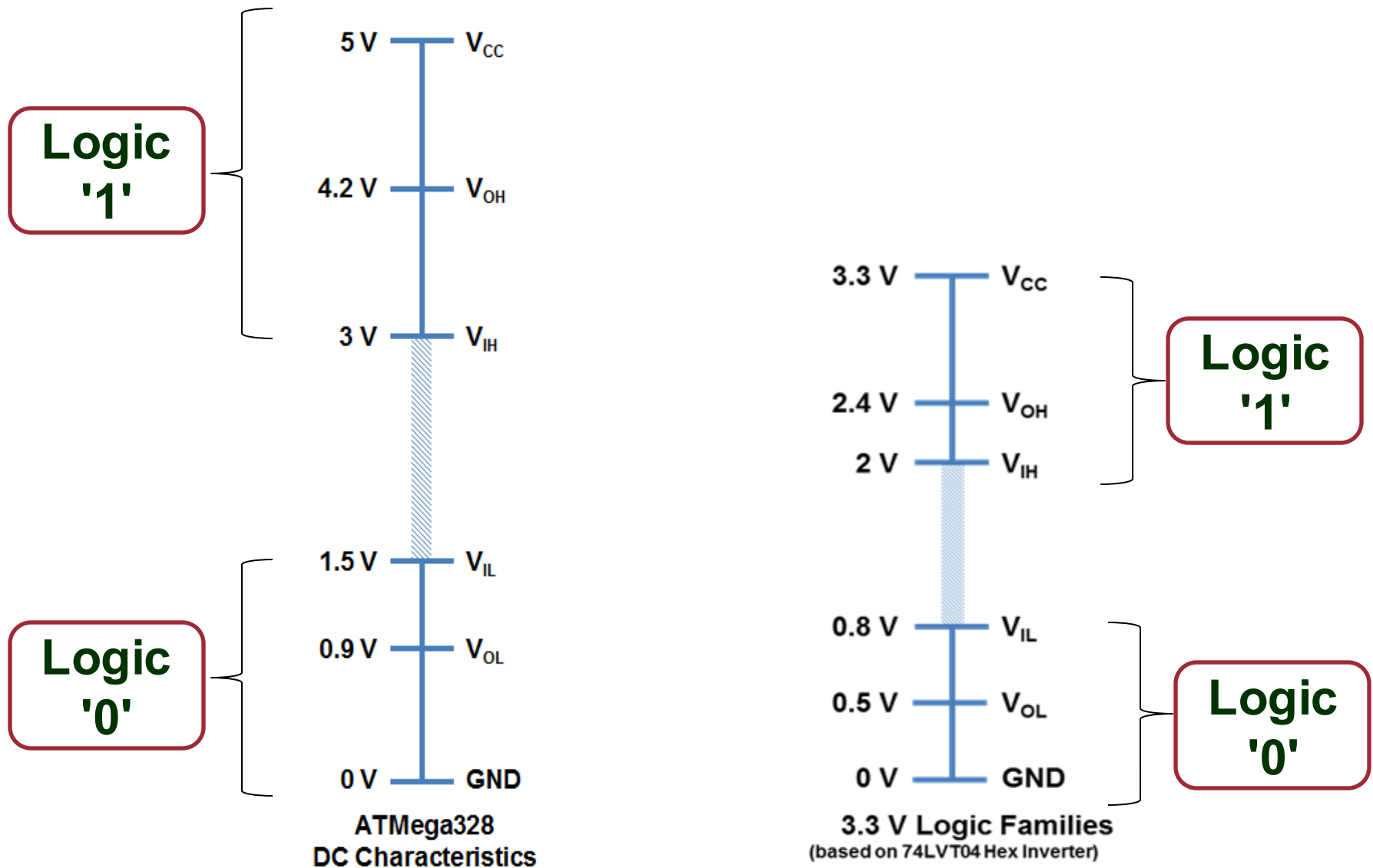
The Easy Way (recommended)

- You can plug your USB from your Arduino into your RPi.
 - ❑ **Problems:** you will need to use Serial 0!

The Harder Way

- You can wire up the UART pins from your Arduino into your RPi.
 - ❑ **Problems:** How to deal with the voltage difference? More wires / points of failure.

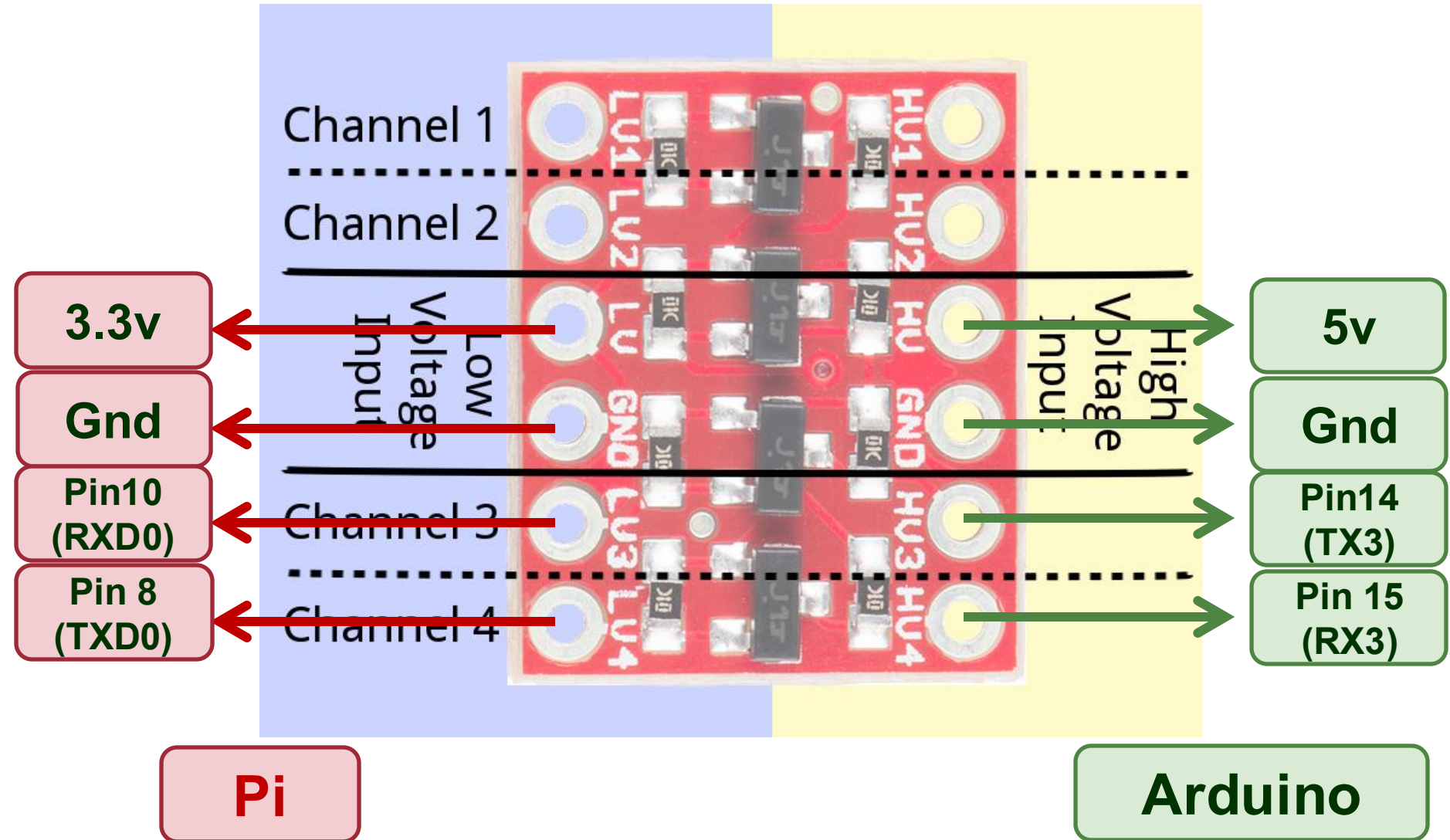
3.3v $\leftrightarrow$ 5v: Is there a problem?



3.3v $\leftrightarrow$ 5v (Level Shifting)

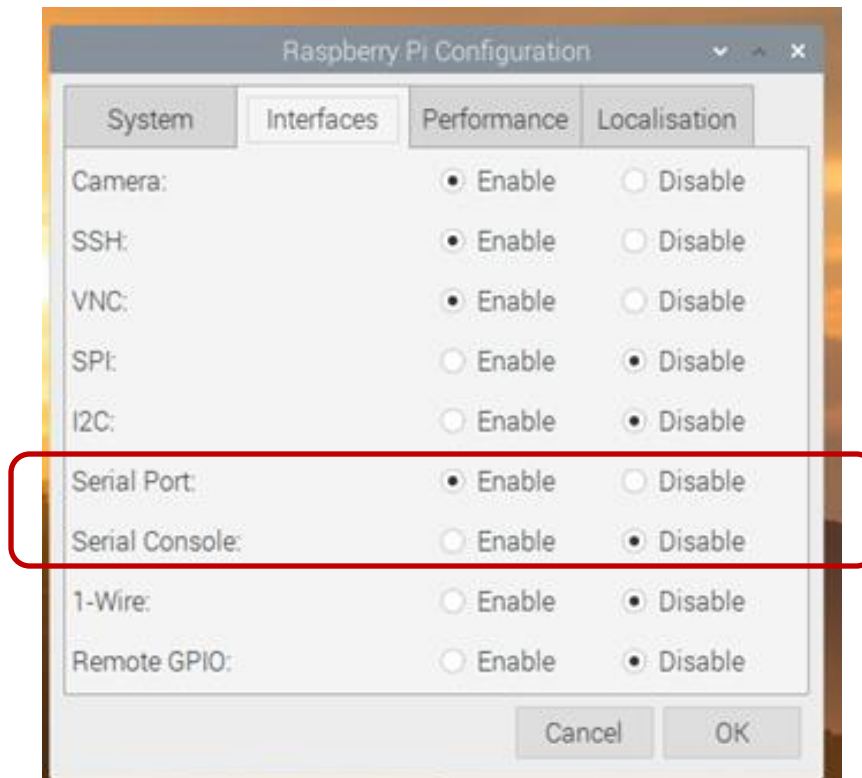
- Ways to shift from **5v** to **3.3v**:
 - **Voltage Divider**
 - **Level shifter** (see next slide)
- Generally, there is no need to shift from 3.3v to 5v
 - 3.3v is above the ~3v “high” level on the Arduino as seen in previous slide
 - Since we have a 4 channels level shifter:
 - We are going to shift 3.3v $\rightarrow$ 5v anyway

Logic Level Converter (**CAREFUL!**)



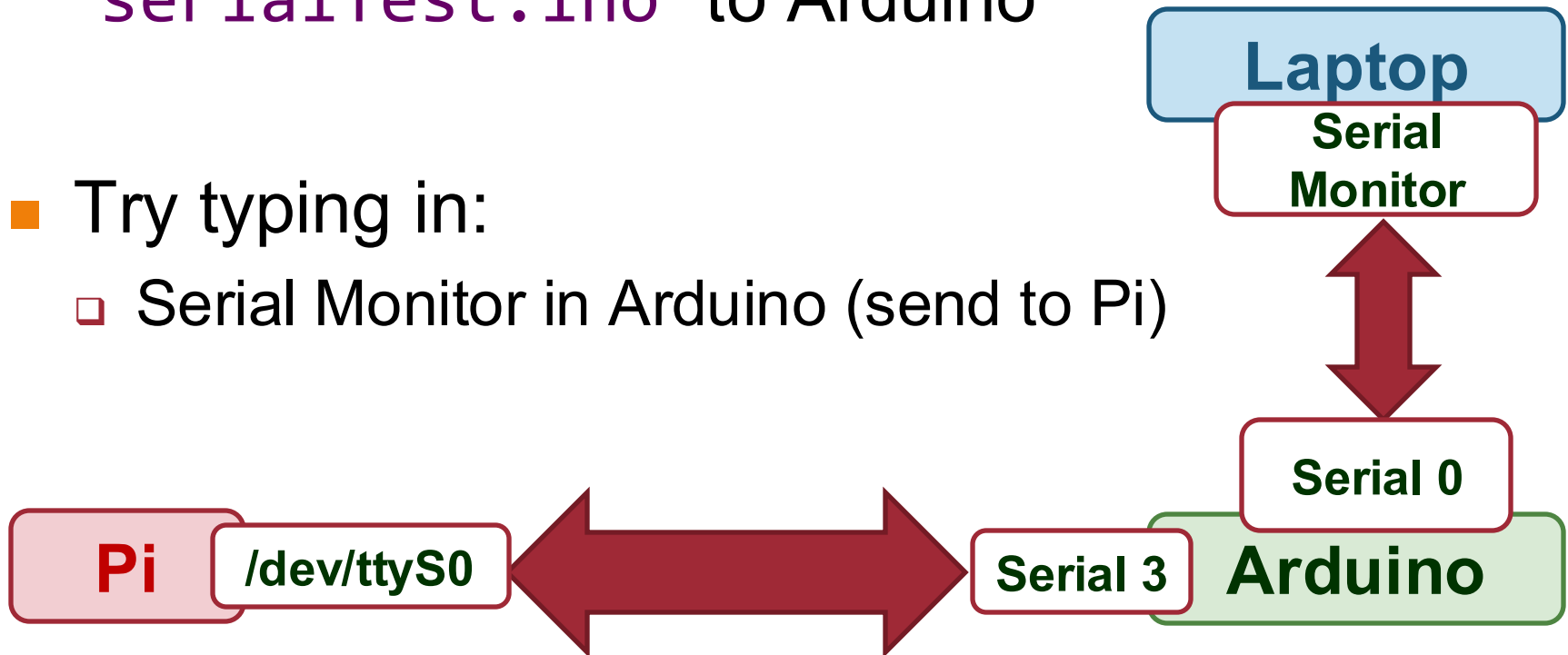
Raspberry Pi: Configuration Needed

- Click on the "Raspberry" button on the top left
 - ❑ → Preferences → Raspberry Pi Configuration
 - ❑ Check the "Interfaces" tab



Hello There!

- **[Pi]** Transfer (or enter) the sample code "`serialTest.py`" to Pi and execute
- **[Arduino]** Upload the sample code "`serialTest.ino`" to Arduino
- Try typing in:
 - Serial Monitor in Arduino (send to Pi)



I say this, then you say that, ok?

SERIAL PROTOCOL DESIGN

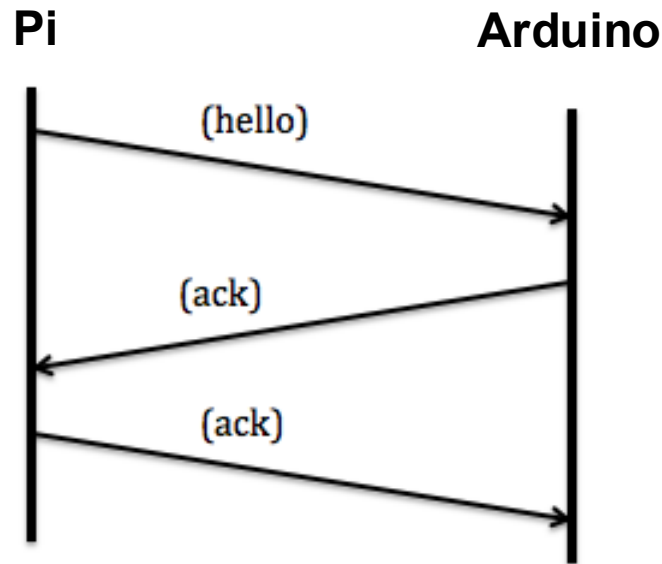
Decisions to make

1. Decide on physical connection:
 - ❑ Which UART to use on the Mega?
2. Decide baud rate (standard is 9600)
3. Decide a communication protocol:
 - ❑ Boot Up Handshake
 - ❑ Push / Pull Model
 - ❑ Data packet format

Boot Up 3-Way Handshake

- Objective:

- Allow Pi and Arduino to know that the other party is ready to communicate:



- Do this at the very start of your programs on both RPi and Mega

Let's Shake Hands!

- Implement the 3-way Handshake



- [Pi] Write a "**Handshake.py**":
 - ❑ Use `serial.read()`, `serial.write()` for byte sending/receiving
 - ❑ Print ("Arduino Online") if handshake is successful
- [Arduino] Write to "**Handshake.ino**":
 - ❑ Similarly use `serial.read()`, `serial.write()`
- Definition:
 - ❑ You can define the "Hello" / "ACK" accordingly

BASIC COMMUNICATION MODELS

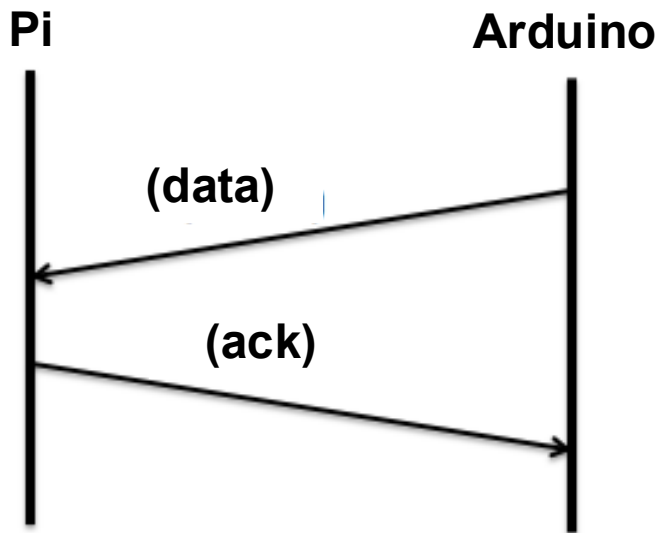
Arduino: Push Data

- Arduino sends data whenever it is available

- Pi monitors and buffers data as it comes in:

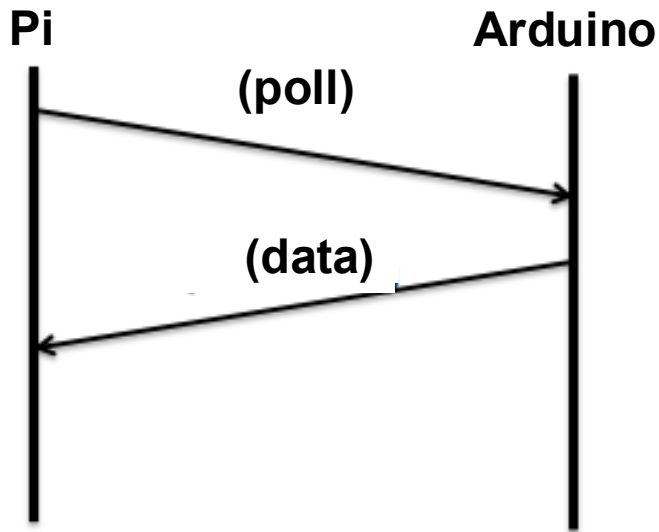
+Arduino sends data whenever it is available

- Pi needs to buffer incoming data



Pi: Pull (Poll) Data

- Arduino waits for poll request from Pi



- Pi requests data when it needs it

+ Pi decides when it needs the data and sends poll packet

- If Pi doesn't poll often enough, may lose data on Arduino

Command / Response Format

- You need to decide the command / response format between Pi / Arduino
- Below is a simple sketch:

Command	Command Symbol	Data	Meaning of Data
Forward	'F'	Integer, D	Distance to move
Backward	...	...	...
Left	...	...	...
Right	...	...	...
Turn To	...	...	...
Obstacle?	...	...	...

Design Your Protocol

- Discuss with your team (including your friends in the A track)
- Come up with a simple sketch of your command / response list
- There is no time limit for this task
 - ❑ You probably need to change this protocol a few times before the end of project
 - ❑ Remember to **document your design**, e.g. using table as shown in previous slide

Remaining Issues - Camera

How do we stream video from our Rpi to our computer?

It is up to you! We need low latency for driving!

- MQTT? Possible but slow! Good for DL perhaps?

- Flask/OpenCV? Also possible, many options.

For example: <https://github.com/EbenKouao/pi-camera-stream-flask>

- We can look towards video streaming technologies. One possibility is mjpg-streamer

To install on the Rpi: `sudo apt-get install mjpg-streamer`

Then run: `mjpg_streamer -i "input_raspicam.so" -o "output_http.so -w ./www"`

Finally point your browser to `http://<raspberrypi_ip>:8080`

Remaining Issues – Remote Control

How do we control our robot from our laptop? We want to use WiFi ofcourse, but how do we build such a system.

We recommend you write a python script on your laptop to communicate over WiFi to your serial python script on the Rpi.

But there are many possibilities

- ❑ Flask? Django?
- ❑ Raw TCP/IP socket?

On your Pi:

```
from flask import Flask, request
```

```
app = Flask(__name__)
```

```
@app.route('/control', methods=['POST'])
```

```
def control():
```

```
    command = request.form['command']
```

```
    # Add logic to control the robot based on the  
command
```

```
    # e.g., if command == 'forward':
```

```
move_robot_forward()
```

```
    return 'Command received: ' + command
```

```
if __name__ == '__main__':
```

```
    app.run(host='0.0.0.0', port=5000)
```

On your Laptop:

**Use the restful api:
Or you can even create a html
form**

```
<form action="http://<microcontroller_ip>:5000/control"
method="post">
    <button name="command"
value="forward">Forward</button>
    <button name="command"
value="backward">Backward</button>
    <!-- Add more buttons as needed -->
</form>
```

Your Tasks for Today

- Get your Arduino and Pi communicating
- Implement and test the 3-way Handshake
- Think about your protocol and try designing it!
- Video streaming / camera
- Remote control

The Weekend!

- Finish/complete your robot chassis (follow the instruction sheet). You can do whatever design you want. However, it must have:
 - ❑ Battery pack and Powerbank
 - ❑ Arduino Mega
 - ❑ Motor shield and all four motors
 - ❑ RPi and RPi Camera
- Your robot should be able to drive forward, backward, and turn left and right. Can you drive it remotely? (You can use VNC/ssh to connect to the RPi and then use the RPi can connect over serial to the Arduino to begin with)
- Your robot should be able to take a photo!



END

